

ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ

з дисципліни «Основи програмування на мові Javascript»

Студент групи ПЗ-23-1

Бажан Денис Сергійович

кафедра комп'ютерних технологій, ДНУ

2024/2025

1. Постановка задачі

У рамках лабораторної роботи було поставлено три основні завдання:

1. Виконати перестановку значень п'яти змінних за заданою схемою ACDBE.
2. Обчислити периметри трикутника та прямокутника, використовуючи координати вершин.
3. Перевірити, чи точка з заданими координатами знаходиться всередині прямокутника.

Метою роботи є освоєння основних принципів програмування на мові Javascript, зокрема роботи з масивами, функціями, умовними операторами та математичними обчисленнями.

2. Опис розв'язку

Для розв'язання поставлених завдань було розроблено програму, що складається з трьох функцій:

1. **Функція переставитиЗначення():**
 - а. Використовує деструктуризацію для перестановки значень змінних відповідно до схеми ACDBE.
2. **Функція обчислитиПериметри():**
 - а. Обчислює периметр трикутника, використовуючи координати трьох точок. Для цього була застосована формула відстані між двома точками на площині.
 - б. Також обчислюється периметр прямокутника за допомогою відомої формули для периметра прямокутника.
3. **Функція перевіритиТочку(x, y):**

- а. Перевіряє, чи точка з координатами (x, y) знаходиться всередині прямокутника з координатами верхнього лівого кута (2, 6) і нижнього правого кута (6, 2).

Програма містить функції, що виконують розрахунки, та тестують їх за допомогою різних наборів значень.

3. Вихідний текст програми розв'язку задачі (основні фрагменти з коментарями)

```
// Функція для перестановки значень змінних за схемою ACDBE
function переставитиЗначення() {
    let A = 1, B = 2, C = 3, D = 4, E = 5;
    [A, B, C, D, E] = [C, D, B, A, E]; // Перестановка значень
    return { A, B, C, D, E };
}

// Функція для обчислення периметрів трикутника та прямокутника
function обчислитиПериметри() {
    let [x1, y1] = [0, 0];
    let [x2, y2] = [3, 0];
    let [x3, y3] = [0, 4];

    // Обчислення довжин сторін трикутника
    const сторона = (x1, y1, x2, y2) => Math.sqrt((x2 - x1) ** 2 +
(y2 - y1) ** 2);
    let a = сторона(x1, y1, x2, y2);
    let b = сторона(x2, y2, x3, y3);
    let c = сторона(x3, y3, x1, y1);
    let периметрТрикутника = a + b + c;

    // Прямокутник
    let ширина = 5, висота = 3;
    let периметрПрямокутника = 2 * (ширина + висота);

    return { периметрТрикутника: периметрТрикутника.toFixed(2),
периметрПрямокутника };
}

// Функція для перевірки, чи точка знаходиться всередині
прямокутника
function перевіритиТочку(x, y) {
```

```

    let [x1, y1] = [2, 6]; // Верхній лівий кут
    let [x2, y2] = [6, 2]; // Нижній правий кут
    let перетинає = x >= x1 && x <= x2 && y >= y2 && y <= y1;
    return перетинає;
}

// Тестування всіх функцій
function тест() {
    console.log("=== Тестування ===");
    let значення = переставитиЗначення();
    console.log("1) Перестановка змінних:", значення);

    let периметри = обчислитиПериметри();
    console.log("2) Периметри фігур:", периметри);

    let точка = [4, 4];
    console.log(`3) Точка (${точка[0]}, ${точка[1]}) знаходиться
всередині прямокутника:`, перевіритиТочку(...точка));
}

// Запуск тесту
тест();

```

4. Опис інтерфейсу програми (керівництво користувача)

Програма автоматично виконує три основні завдання:

1. Виводить результат перестановки значень змінних.
2. Обчислює периметри трикутника та прямокутника і виводить їх в консоль.
3. Перевіряє, чи точка з заданими координатами знаходиться всередині прямокутника.

Всі результати тестів виводяться в консоль. Для запуску тестування достатньо викликати функцію `тест()`.

5. Опис тестових прикладів

1. Перестановка змінних:

- а. Вхід: A = 1, B = 2, C = 3, D = 4, E = 5.
- б. Вихід: A = 3, B = 4, C = 1, D = 2, E = 5.

2. Периметри фігур:

- а. Трикутник з координатами вершин $(0, 0)$, $(3, 0)$, $(0, 4)$ має периметр 12.00.
- б. Прямокутник з шириною 5 та висотою 3 має периметр 16.

3. Перевірка точки:

- а. Точка з координатами $(4, 4)$ знаходиться всередині прямокутника.

6. Аналіз помилок (опис усунення зауважень)

Під час тестування програма працювала коректно і не містила критичних помилок. У разі наявності помилок, що могли б виникнути, можливе додавання перевірок на коректність введених даних для функції перевіритиТочку, щоб забезпечити правильне виконання програми за будь-яких умов.